Developer Guide

Prerequisites

Tool	Version	Install
Docker Desktop	Latest	docker.com
Java JDK 17	17 LTS	`brew install openjdk@17`
Maven	3.9+	`brew install maven`
Git	Any	git-scm.com
AWS CLI	v2	`pip install awscli`

Day-to-Day Workflow

1. Start Working on a Feature

```
# Always branch off develop, never off main
git checkout develop
git pull
git checkout -b feature/add-coupon-codes
```

2. Make Changes and Test Locally

```
# Start all infrastructure
docker compose up -d


Run tests for the service you changed


cd order-service
mvn test
```

```
Test your endpoint manually

curl -X POST http://localhost:8000/api/orders ...
```

3. Push and Open a PR

```
git push -u origin feature/add-coupon-codes
```

```
Open PR on GitHub → targeting develop branch
```

What happens automatically when you open the PR:

- ✅ All 6 services' unit tests run (30 tests)
- ✅ JaCoCo coverage check (minimum 50%)
- ✅ CodeQL SAST scan (security analysis)
- ✅ PR cannot merge until tests pass

4. After PR is Reviewed and Merged to `develop`

What happens automatically:

- ✅ Tests run again
- ✅ Docker images built (all 6 services)
- ✅ Trivy vulnerability scan
- ✅ Images pushed to Floci ECR
- ✅ Full stack deployed via docker-compose
- ✅ Smoke test: register user → create product → place order
- ✅ Slack/email notification (if configured)

5. Promote to Production (merge develop → main)

```
# Open PR: develop → main on GitHub
```

```
After review + approval → merge
```

What happens:

- Same pipeline as develop
- PAUSES for manual approval (production GitHub Environment)
- Approver clicks "Approve deployment" in GitHub
- Then deploys

Branch Rules

```
main      ← production, protected, requires PR + approval
develop   ← staging, protected, requires PR
feature/* ← your work, branch off develop
hotfix/*  ← urgent fixes, branch off main
```

Never push directly to `main` or `develop`.

Running Services Locally (without Docker)

If you want to run services with hot-reload for development:

```
# Terminal 1: Start infrastructure only
docker compose up -d postgres-user postgres-product postgres-order \
  postgres-inventory postgres-payment redis kafka kafka-init kong floci

# Terminal 2+: Run a specific service
cd user-service
mvn spring-boot:run
```

```
Changes reload automatically with Spring DevTools
```

Environment Variables

All configuration is in `docker-compose.yml` for local development.

For K8s, see `k8s/services/configmap.yaml` and `k8s/services/secrets.example.yaml`.

Key values for local:

```
SPRING_DATA_REDIS_PASSWORD=redis_pass
SPRING_KAFKA_BOOTSTRAP_SERVERS=kafka:9092
```

```
DB credentials: see docker-compose.yml per service
```

Running the AWS Simulation (Floci)

```
# Configure AWS CLI to point to Floci
export AWS_ENDPOINT_URL=http://localhost:4566
export AWS_DEFAULT_REGION=ap-south-1
export AWS_ACCESS_KEY_ID=test
export AWS_SECRET_ACCESS_KEY=test

# Now any aws CLI command goes to Floci instead o
f real AWS

aws s3 ls
aws ecr describe-repositories
aws eks list-clusters
```

Common Commands

```
# Check all pods in K8s
kubectl get pods -n ecommerce

# Check logs of a service

kubectl logs -n ecommerce -l app=order-service --tail=50
```

Check Kafka consumer lag

```
docker exec kafka kafka-consumer-groups \
  --bootstrap-server kafka:9092 \
  --describe --group payment-service-group
```

Connect to a database directly

```
docker exec -it postgres-order psql -U order_svc -d orderdb
```

Check Redis cache

```
docker exec redis redis-cli -a redis_pass KEYS '*'
```